29. Goroutines and the Scheduler

29. Goroutines and the Scheduler

Phase: Phase 5 — Concurrency
Estimated time: 1–2 days
Slug: `29-goroutines`

Learning objectives

- Start goroutines with go
- Explain M:N scheduling at a high level
- Avoid leaking goroutines
- Synchronize completion with WaitGroup

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

Goroutines are cheap concurrent functions scheduled by the Go runtime. They are why Go shines for servers and pipelines — used carefully.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- `go f()` starts a goroutine; it may outlive the caller.
- The scheduler multiplexes goroutines onto OS threads.
- Always have an exit condition — leaks are real.
- sync.WaitGroup waits for a set of goroutines.
- Do not share mutable state without sync (next lessons).

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-goroutines && cd /tmp/gw-goroutines
go mod init example.com/scratch
```

Then create `main.go`:

```
package main
```

```
import (
    "fmt"
    "sync"
)

func main() {
    var wg sync.WaitGroup
    for i := 0; i < 3; i++ {
        wg.Add(1)
        go func(n int) {
            defer wg.Done()
            fmt.Println("hi", n)
        }(i)
    }
    wg.Wait()
}
```

Run it:

```
go run .
```

Expected output:

```
hi 0
hi 1
hi 2
(order may vary)
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Launch N goroutines with WaitGroup.
2. Forget Done once and observe a hang — then fix.
3. Pass loop variable as an argument (classic gotcha in older Go).

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

Goroutines

```
go func() {
    doWork()
}()
```

Goroutines are lightweight concurrent functions multiplexed onto OS threads by the Go scheduler (M:N).

Leaks

A goroutine leak is a goroutine that never exits — common when blocked on channel/IO forever. Always plan an exit path (context, close channel, timeout).

WaitGroup

```
var wg sync.WaitGroup
wg.Add(1)
go func() {
    defer wg.Done()
    work()
}()
wg.Wait()
```

Do not assume goroutine order. Concurrency ≠ parallelism (but Go gives you both tools).

Failure modes to watch while coding

- Fire-and-forget goroutines in request handlers without limits
- Capturing loop variables incorrectly (modern Go fixed per-iteration vars; still be careful)
- Waiting forever with no timeout

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- threading.Thread is heavy; goroutines are lighter.
- asyncio is cooperative; Go is concurrent with a scheduler.
- GIL limits CPU threads; Go uses multiple OS threads freely.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would “work”. The goal is a Go-shaped intuition.

Going deeper

Structure concurrency as trees of tasks with clear cancellation (context).

Practice checklist

Before the exercise, confirm you can:

1. Explain “Goroutines and the Scheduler” aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson’s rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- Fire-and-forget goroutines in request handlers without limits
- Capturing loop variables incorrectly (modern Go fixed per-iteration vars; still be careful)
- Waiting forever with no timeout

Exercises

Work in `exercises/29-goroutines/`. Read that folder's `README.md` before you code.

What you will build

Use WaitGroup (or the exercise API) so all goroutines finish before return.

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/29-goroutines/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. What does the go keyword do?
2. What is a goroutine leak?
3. Why WaitGroup?

Further reading

- https://go.dev/tour/concurrency
- https://go.dev/blog/scheduler